

Hướng Dẫn Toàn Diện về Xây Dựng Skill cho Claude.ai

(bản tiếng Việt)

Mục Lục

Giới Thiệu

Chương 1 – Kiến Thức Cơ Bản

Chương 2 – Lập Kế Hoạch và Thiết Kế

Chương 3 – Kiểm Thử và Cải Tiến

Chương 4 – Phân Phối và Chia Sẻ

Chương 5 – Mô Hình và Xử Lý Sự Cố

Chương 6 – Tài Nguyên và Tham Khảo

Giới Thiệu

Skill là một tập hợp các hướng dẫn — được đóng gói dưới dạng một thư mục đơn giản — giúp Claude học cách xử lý các nhiệm vụ hoặc quy trình công việc cụ thể. Skill là một trong những cách mạnh mẽ nhất để tùy chỉnh Claude cho nhu cầu riêng của bạn. Thay vì phải giải thích lại sở thích, quy trình và chuyên môn trong từng cuộc hội thoại, skill cho phép bạn chỉ cần dạy Claude một lần và được hưởng lợi mỗi khi sử dụng.

Skill phát huy sức mạnh khi bạn có các quy trình lặp đi lặp lại: tạo giao diện frontend từ đặc tả, thực hiện nghiên cứu với phương pháp nhất quán, tạo tài liệu theo phong cách của nhóm bạn, hoặc điều phối các quy trình nhiều bước. Chúng hoạt động tốt với các khả năng tích hợp sẵn của Claude như thực thi mã và tạo tài liệu. Với những ai đang xây dựng tích hợp MCP, skill bổ sung thêm một lớp mạnh mẽ — giúp biến quyền truy cập công cụ thô thành các quy trình đáng tin cậy, được tối ưu hóa.

Hướng dẫn này bao gồm mọi thứ bạn cần biết để xây dựng các skill hiệu quả — từ lập kế hoạch và cấu trúc đến kiểm thử và phân phối. Dù bạn đang xây dựng skill cho bản thân, nhóm của mình, hay cộng đồng, bạn sẽ tìm thấy các mô hình thực tiễn và ví dụ thực tế xuyên suốt tài liệu này.

Bạn sẽ học được gì:

- Yêu cầu kỹ thuật và các phương pháp tốt nhất cho cấu trúc skill
 - Các mô hình cho skill độc lập và quy trình được tăng cường bởi MCP
 - Các mô hình đã được chứng minh hoạt động tốt trong nhiều trường hợp sử dụng khác nhau
- Cách kiểm thử, cải tiến và phân phối skill của bạn

Đối tượng hướng đến:

- Nhà phát triển muốn Claude tuân theo các quy trình công việc cụ thể một cách nhất quán
- Người dùng nâng cao muốn Claude tuân theo các quy trình công việc cụ thể
- Các nhóm muốn chuẩn hóa cách Claude hoạt động trong toàn tổ chức

Hai Con Đường Trong Hướng Dẫn này

Đang xây dựng skill độc lập? Tập trung vào Kiến Thức Cơ Bản, Lập Kế Hoạch và Thiết Kế, và danh mục 1–2. Đang tăng cường tích hợp MCP? Phần "Skill + MCP" và danh mục 3 dành cho bạn. Cả hai con đường đều có chung yêu cầu kỹ thuật — bạn chỉ cần chọn phần phù hợp với trường hợp của mình.

Điều bạn sẽ đạt được sau hướng dẫn này: Đến cuối tài liệu, bạn sẽ có thể xây dựng một skill hoạt động được trong một buổi làm việc. Hãy dự tính khoảng 15–30 phút để xây dựng và kiểm thử skill đầu tiên của bạn bằng skill-creator.

Hãy bắt đầu nào.

Chương 1 – Kiến Thức Cơ Bản

Skill là gì?

Một skill là một thư mục chứa:

- SKILL.md (bắt buộc): Hướng dẫn dạng Markdown với YAML frontmatter
- scripts/ (tùy chọn): Mã thực thi (Python, Bash, v.v.)
- references/ (tùy chọn): Tài liệu được tải khi cần
- assets/ (tùy chọn): Template, font chữ, biểu tượng được sử dụng trong đầu ra

Các nguyên tắc thiết kế cốt lõi

Hiện Thị Dần Dần (Progressive Disclosure)

Skill sử dụng hệ thống ba cấp:

- Cấp 1 (YAML frontmatter): Luôn được tải vào system prompt của Claude. Cung cấp vừa đủ thông tin để Claude biết khi nào mỗi skill nên được sử dụng mà không cần tải toàn bộ vào ngữ cảnh.
- Cấp 2 (Nội dung SKILL.md): Được tải khi Claude thấy skill liên quan đến nhiệm vụ hiện tại. Chứa hướng dẫn và chỉ dẫn đầy đủ.
- Cấp 3 (Các tệp liên kết): Các tệp bổ sung đi kèm trong thư mục skill mà Claude có thể điều hướng và khám phá khi cần thiết.

Cơ chế hiển thị dần dần này giảm thiểu việc sử dụng token trong khi vẫn duy trì chuyên môn chuyên biệt.

Khả Năng Kết Hợp (Composability)

Claude có thể tải nhiều skill cùng lúc. Skill của bạn nên hoạt động tốt cùng với các skill khác — không được giả định rằng đây là khả năng duy nhất hiện có.

Tính Di Động (Portability)

Skill hoạt động giống hệt nhau trên Claude.ai, Claude Code và API. Tạo một skill một lần và nó hoạt động trên mọi nền tảng mà không cần chỉnh sửa, miễn là môi trường hỗ trợ các phụ thuộc mà skill yêu cầu.

Dành Cho Nhà Phát Triển MCP: Skill + Connector

□□ Đang xây dựng skill độc lập không có MCP? Bỏ qua và đến phần Lập Kế Hoạch và Thiết Kế — bạn có thể quay lại đây sau.

Nếu bạn đã có MCP server hoạt động, bạn đã hoàn thành phần khó nhất. Skill là lớp kiến thức bên trên — ghi lại các quy trình và các thông lệ tốt nhất mà bạn đã biết, để Claude có thể áp dụng chúng một cách nhất quán.

Phép So Sánh Nhà Bếp

MCP cung cấp "nhà bếp chuyên nghiệp": quyền truy cập vào công cụ, nguyên liệu và thiết bị.

Skill cung cấp "công thức nấu ăn": hướng dẫn từng bước về cách tạo ra thứ gì đó có giá trị.

Cùng nhau, chúng cho phép người dùng hoàn thành các nhiệm vụ phức tạp mà không cần phải tự tìm hiểu từng bước.

Cách chúng hoạt động cùng nhau:

MCP (Kết nối)	Skill (Kiến thức)
Kết nối Claude với dịch vụ của bạn (Notion, Asana, Linear, v.v.)	Dạy Claude cách sử dụng dịch vụ của bạn hiệu quả
Cung cấp quyền truy cập dữ liệu thời gian thực và gọi công cụ	Ghi lại các quy trình và thông lệ tốt nhất

Tại sao điều này quan trọng với người dùng MCP của bạn

Không có skill:

- Người dùng kết nối MCP nhưng không biết phải làm gì tiếp theo
- Phiếu hỗ trợ hỏi "tôi làm X với tích hợp của bạn như thế nào"
- Mỗi cuộc hội thoại bắt đầu từ đầu
- Kết quả không nhất quán vì người dùng đặt câu hỏi theo nhiều cách khác nhau
- Người dùng đổ lỗi cho connector trong khi vấn đề thực sự là thiếu hướng dẫn quy trình

Với skill:

- Các quy trình được xây dựng sẵn tự động kích hoạt khi cần
- Sử dụng công cụ nhất quán, đáng tin cậy
- Thực tiễn tốt nhất được nhúng trong mỗi tương tác
- Giảm độ dốc học tập cho tích hợp của bạn

Chương 2 – Lập Kế Hoạch và Thiết Kế

Bắt đầu với các trường hợp sử dụng

Trước khi viết bất kỳ mã nào, hãy xác định 2–3 trường hợp sử dụng cụ thể mà skill của bạn cần hỗ trợ.

Ví dụ về định nghĩa trường hợp sử dụng tốt:

Trường Hợp Sử Dụng: Lập Kế Hoạch Sprint Dự Án

Kích hoạt: Người dùng nói "giúp tôi lập kế hoạch sprint này" hoặc "tạo task sprint"

Các bước:

1. Lấy trạng thái dự án hiện tại từ Linear (qua MCP)
2. Phân tích vận tốc và năng lực nhóm
3. Gợi ý ưu tiên hóa task
4. Tạo task trong Linear với nhãn và ước tính phù hợp

Kết quả: Sprint được lập kế hoạch đầy đủ với các task đã tạo

Hãy tự hỏi bản thân:

- Người dùng muốn hoàn thành điều gì?
- Quy trình nhiều bước nào cần thiết?
- Công cụ nào được cần (tích hợp sẵn hay MCP)?
- Kiến thức chuyên môn hoặc thực tiễn tốt nhất nào nên được nhúng vào?

Các danh mục trường hợp sử dụng skill phổ biến

Tại Anthropic, chúng tôi đã quan sát thấy ba danh mục trường hợp sử dụng phổ biến:

Danh mục 1: Tạo Tài Liệu & Tài Sản

Dùng cho: Tạo đầu ra nhất quán, chất lượng cao bao gồm tài liệu, bài thuyết trình, ứng dụng, thiết kế, mã, v.v.

Ví dụ thực tế: skill frontend-design (cũng xem các skill cho docx, pptx, xlsx và ppt)

"Tạo giao diện frontend đặc biệt, sẵn sàng production với chất lượng thiết kế cao. Sử dụng khi xây dựng web component, trang, artifact, poster, hoặc ứng dụng."

Các kỹ thuật chính:

- Hướng dẫn phong cách và tiêu chuẩn thương hiệu được nhúng sẵn
- Cấu trúc template cho đầu ra nhất quán
- Danh sách kiểm tra chất lượng trước khi hoàn thiện
- Không yêu cầu công cụ bên ngoài — sử dụng các khả năng tích hợp sẵn của Claude

Danh mục 2: Tự Động Hóa Quy Trình

Dùng cho: Các quy trình nhiều bước được hưởng lợi từ phương pháp nhất quán, bao gồm điều phối qua nhiều MCP server.

Ví dụ thực tế: skill skill-creator

"Hướng dẫn tương tác để tạo skill mới. Dẫn dắt người dùng qua định nghĩa trường hợp sử dụng, tạo frontmatter, viết hướng dẫn và xác thực."

Các kỹ thuật chính:

- Quy trình từng bước với các cổng xác thực
- Template cho các cấu trúc phổ biến
- Đề xuất xem xét và cải tiến tích hợp sẵn
- Vòng lặp cải tiến liên tục

Danh mục 3: Tăng Cường MCP

Dùng cho: Hướng dẫn quy trình để tăng cường quyền truy cập công cụ mà một MCP server cung cấp.

Ví dụ thực tế: skill sentry-code-review (từ Sentry)

"Tự động phân tích và sửa các lỗi được phát hiện trong GitHub Pull Request bằng cách sử dụng dữ liệu giám sát lỗi của Sentry qua MCP server của họ."

Các kỹ thuật chính:

- Điều phối nhiều lệnh gọi MCP theo thứ tự
- Nhúng kiến thức chuyên môn lĩnh vực
- Cung cấp ngữ cảnh mà người dùng nếu không sẽ phải tự chỉ định
- Xử lý lỗi cho các vấn đề MCP phổ biến

Xác Định tiêu chí thành công

Bạn sẽ biết skill của mình đang hoạt động như thế nào? Đây là các mục tiêu kỳ vọng — các điểm chuẩn tương đối hơn là ngưỡng chính xác. Hướng đến sự nghiêm ngặt nhưng chấp nhận rằng sẽ có yếu tố đánh giá chủ quan. Anthropic đang tích cực phát triển hướng dẫn và công cụ đo lường mạnh mẽ hơn.

Chỉ số định lượng:

- Skill kích hoạt trên 90% truy vấn liên quan

→ Cách đo: Chạy 10–20 truy vấn thử nghiệm cần kích hoạt skill. Theo dõi số lần skill tự động tải so với cần kích hoạt thủ công.

- Hoàn thành quy trình trong X lệnh gọi công cụ

→ Cách đo: So sánh cùng một nhiệm vụ có và không có skill. Đếm lệnh gọi công cụ và tổng token tiêu thụ.

- 0 lệnh gọi API thất bại mỗi quy trình
- Cách đo: Giám sát log MCP server trong quá trình chạy thử nghiệm. Theo dõi tỷ lệ thử lại và mã lỗi.

Chỉ số định tính:

- Người dùng không cần nhắc Claude về các bước tiếp theo
- Cách đánh giá: Trong khi kiểm thử, ghi lại tần suất bạn cần điều chỉnh hoặc làm rõ. Xin phản hồi từ người dùng beta.
- Quy trình hoàn thành mà không cần người dùng sửa
- Cách đánh giá: Chạy cùng một yêu cầu 3–5 lần. So sánh đầu ra về tính nhất quán cấu trúc và chất lượng.
- Kết quả nhất quán qua các phiên
- Cách đánh giá: Người dùng mới có thể hoàn thành nhiệm vụ lần đầu tiên với hướng dẫn tối thiểu không?

Yêu cầu kỹ thuật

Cấu trúc tệp

```
your-skill-name/  
├── SKILL.md # Bắt buộc - tệp skill chính  
├── scripts/ # Tùy chọn - mã thực thi  
│   ├── process_data.py  
│   └── validate.sh  
├── references/ # Tùy chọn - tài liệu  
│   ├── api-guide.md  
│   └── examples/  
├── assets/ # Tùy chọn - template, v.v.  
└── report-template.md
```

Các quy tắc quan trọng

Đặt tên SKILL.md:

- Phải là chính xác SKILL.md (phân biệt chữ hoa/thường)
- Không chấp nhận các biến thể (SKILL.MD, skill.md, v.v.)

Đặt tên thư mục skill:

- Dùng kebab-case: notion-project-setup 
- Không dùng khoảng cách: Notion Project Setup 
- Không dùng gạch dưới: notion_project_setup 
- Không dùng chữ hoa: NotionProjectSetup 

Không có README.md:

- Không thêm README.md vào thư mục skill
- Tất cả tài liệu đặt trong SKILL.md hoặc references/
- Lưu ý: khi phân phối qua GitHub, bạn vẫn muốn có README ở cấp repository để người đọc có thể hiểu — xem Phân Phối và Chia Sẻ.

YAML Frontmatter: Phần quan trọng nhất

YAML frontmatter là cách Claude quyết định có tài skill của bạn hay không. Hãy làm đúng phần này.

Định dạng tối thiểu bắt buộc:

```
---
name: your-skill-name
description: Mô tả chức năng. Sử dụng khi người dùng hỏi về [cụm từ cụ thể]. --
-
```

Yêu cầu trường:

name (bắt buộc):

- Chỉ dùng kebab-case
- Không có khoảng cách hoặc chữ hoa
- Nên khớp với tên thư mục

description (bắt buộc):

- PHẢI bao gồm CẢ HAI: Skill làm gì + Khi nào sử dụng (điều kiện kích hoạt) •
- Dưới 1024 ký tự
- Không có thẻ XML (< hoặc >)
- Bao gồm các nhiệm vụ cụ thể mà người dùng có thể nói
- Đề cập đến loại tệp nếu liên quan

license (tùy chọn):

- Dùng nếu muốn skill mã nguồn mở
- Phổ biến: MIT, Apache-2.0

compatibility (tùy chọn):

- 1-500 ký tự
- Chỉ ra yêu cầu môi trường: ví dụ sản phẩm dự định, gói hệ thống cần thiết, nhu cầu truy cập mạng, v.v.

metadata (tùy chọn):

- Bất kỳ cặp key-value tùy chỉnh nào
- Gợi ý: author, version, mcp-server

```
metadata:
  author: ProjectHub
  version: 1.0.0
  mcp-server: projecthub
```

Hạn chế bảo mật

Bị cấm trong frontmatter:

- Dấu ngoặc nhọn XML (< >)
- Skill có tên chứa "claude" hoặc "anthropic" (đã được đặt trước)

Lý do: Frontmatter xuất hiện trong system prompt của Claude. Nội dung độc hại có thể chèn các hướng dẫn.

Viết skill hiệu quả

Trường description

Theo blog kỹ thuật của Anthropic: "Metadata này...cung cấp vừa đủ thông tin để Claude biết khi nào mỗi skill nên được sử dụng mà không cần tải toàn bộ vào ngữ cảnh." Đây là cấp đầu tiên của hiển thị dần dần.

Cấu trúc:

[Chức năng] + [Khi nào sử dụng] + [Khả năng chính]

Ví dụ description tốt:

Tốt - cụ thể và có thể thực hiện được

description: Phân tích file thiết kế Figma và tạo tài liệu handoff cho lập trình viên.

Sử dụng khi người dùng upload file .fig, hỏi về "design specs", "component documentation", hoặc "design-to-code handoff".

Tốt - bao gồm các cụm từ kích hoạt

description: Quản lý quy trình dự án Linear bao gồm lập kế hoạch sprint, tạo task, và theo dõi trạng thái. Sử dụng khi người dùng đề cập đến "sprint", "Linear tasks", "project planning", hoặc hỏi để "create tickets".

Tốt - đề xuất giá trị rõ ràng

description: Quy trình onboard khách hàng đầu cuối cho PayFlow. Xử lý tạo tài khoản,

thiết lập thanh toán, và quản lý đăng ký. Sử dụng khi người dùng nói "onboard new customer", "set up subscription", hoặc "create PayFlow account".

Ví dụ description xấu:

Quá mơ hồ

description: Giúp đỡ với các dự án.

Thiếu kích hoạt

description: Tạo hệ thống tài liệu nhiều trang tĩnh vi.

Quá kỹ thuật, không có kích hoạt người dùng

description: Triển khai mô hình thực thể Project với quan hệ phân cấp.

Viết hướng dẫn chính

Sau frontmatter, viết hướng dẫn thực tế bằng Markdown. Cấu trúc được khuyến nghị:

```
---
name: your-skill
description: [...]
---

# Tên Skill Của Bạn

## Hướng Dẫn

### Bước 1: [Bước Lớn Đầu Tiên]
Giải thích rõ ràng về điều gì xảy ra.

```bash
python scripts/fetch_data.py --project-id PROJECT_ID
Đầu ra mong đợi: [mô tả thế nào là thành công]
```

### Ví Dụ

#### Ví dụ 1: [Tình huống phổ biến]
Người dùng nói: "Thiết lập chiến dịch marketing mới"
Hành động:
1. Lấy các chiến dịch hiện tại qua MCP
2. Tạo chiến dịch mới với các tham số được cung cấp
Kết quả: Chiến dịch được tạo với link xác nhận

### Xử Lý Sự Cố

Lỗi: [Thông báo lỗi phổ biến]
Nguyên nhân: [Tại sao xảy ra]
Giải pháp: [Cách khắc phục]
```

Các thực tiễn tốt nhất cho hướng dẫn

Cụ thể và có thể thực hiện được

✅ Tốt:

Chạy `python scripts/validate.py --input {filename}` để kiểm tra định dạng dữ liệu.

Nếu xác thực thất bại, các vấn đề phổ biến bao gồm:

- Thiếu trường bắt buộc (thêm chúng vào CSV)
- Định dạng ngày không hợp lệ (dùng YYYY-MM-DD)

❌ Không tốt:

Xác thực dữ liệu trước khi tiếp tục.

Tham chiếu rõ ràng đến tài nguyên đi kèm

Trước khi viết truy vấn, tham khảo `references/api-patterns.md` để biết về: -

Hướng dẫn giới hạn tốc độ

- Mô hình phân trang
- Mã lỗi và xử lý

Sử dụng hiển thị dần dần

Giữ SKILL.md tập trung vào hướng dẫn cốt lõi. Chuyển tài liệu chi tiết sang references/ và liên kết đến đó. (Xem Nguyên tắc Thiết Kế Cốt Lõi để hiểu hệ thống ba cấp hoạt động như thế nào.)

Bao gồm xử lý lỗi

Vấn Đề Thường Gặp

Kết Nối MCP Thất Bại

Nếu bạn thấy "Connection refused":

1. Xác minh MCP server đang chạy: Kiểm tra Settings > Extensions
2. Xác nhận API key hợp lệ
3. Thử kết nối lại: Settings > Extensions > [Dịch Vụ Của Bạn] > Reconnect

Chương 3 – Kiểm Thử và Cải Tiến

Skill có thể được kiểm thử ở nhiều mức độ nghiêm ngặt khác nhau tùy theo nhu cầu:

- Kiểm thử thủ công trong Claude.ai — Chạy truy vấn trực tiếp và quan sát hành vi. Lặp nhanh, không cần thiết lập.
- Kiểm thử có kịch bản trong Claude Code — Tự động hóa các trường hợp kiểm thử để xác thực có thể lặp lại qua các thay đổi.
- Kiểm thử lập trình qua Skills API — Xây dựng bộ đánh giá chạy có hệ thống theo các tập kiểm thử được xác định trước.

Chọn cách tiếp cận phù hợp với yêu cầu chất lượng và mức độ hiển thị của skill. Skill được sử dụng nội bộ bởi nhóm nhỏ có nhu cầu kiểm thử khác với skill được triển khai cho hàng nghìn người dùng doanh nghiệp.

□□ Mẹo Pro: Hãy lập trên một nhiệm vụ duy nhất trước khi mở rộng. Chúng tôi nhận thấy rằng những người tạo skill hiệu quả nhất lập trên một nhiệm vụ thách thức duy nhất cho đến khi Claude thành công, sau đó trích xuất phương pháp thành công vào một skill. Điều này tận dụng khả năng học trong ngữ cảnh của Claude và cung cấp tín hiệu nhanh hơn so với kiểm thử rộng. Khi bạn có nền tảng hoạt động, hãy mở rộng sang nhiều trường hợp kiểm thử để đảm bảo phủ sóng.

Cách Tiếp Cận Kiểm Thử Được Khuyến Nghị

Dựa trên kinh nghiệm thực tế, kiểm thử skill hiệu quả thường bao gồm ba lĩnh vực:

1. Kiểm thử kích hoạt

Mục tiêu: Đảm bảo skill tải đúng thời điểm.

Các trường hợp kiểm thử:

-  Kích hoạt với các nhiệm vụ rõ ràng
-  Kích hoạt với các yêu cầu được diễn đạt lại
-  Không kích hoạt với các chủ đề không liên quan

Ví dụ bộ kiểm thử:

Nên kích hoạt:

- "Giúp tôi thiết lập workspace ProjectHub mới"
- "Tôi cần tạo dự án trong ProjectHub"
- "Khởi tạo dự án ProjectHub cho Q4"

KHÔNG nên kích hoạt:

- "Thời tiết ở TP.HCM hôm nay thế nào?"
- "Giúp tôi viết code Python"
- "Tạo bảng tính" (trừ khi skill ProjectHub xử lý sheets)

2. Kiểm thử chức năng

Mục tiêu: Xác minh skill tạo ra đầu ra chính xác.

Các trường hợp kiểm thử:

- Đầu ra hợp lệ được tạo ra
- Các lệnh gọi API thành công
- Xử lý lỗi hoạt động
- Các trường hợp biên được bao phủ

Ví dụ:

Kiểm thử: Tạo dự án với 5 task
Cho trước: Tên dự án "Q4 Planning", 5 mô tả task
Khi: Skill thực thi quy trình
Thì:
- Dự án được tạo trong ProjectHub
- 5 task được tạo với đúng thuộc tính
- Tất cả task liên kết với dự án
- Không có lỗi API

3. So sánh hiệu suất

Mục tiêu: Chứng minh skill cải thiện kết quả so với baseline.

Sử dụng các chỉ số từ phần Xác Định Tiêu Chí Thành Công. Ví dụ so sánh:

Không có skill:
- Người dùng cung cấp hướng dẫn mỗi lần
- 15 lượt trao đổi qua lại
- 3 lệnh gọi API thất bại cần thử lại
- 12.000 token tiêu thụ

Với skill:
- Thực thi quy trình tự động
- Chỉ 2 câu hỏi làm rõ
- 0 lệnh gọi API thất bại
- 6.000 token tiêu thụ

Sử dụng skill skill-creator

Skill skill-creator — có sẵn trong Claude.ai qua thư mục plugin hoặc tải xuống cho Claude Code — có thể giúp bạn xây dựng và lặp lại các skill. Nếu bạn có MCP server và biết 2-3 quy trình hàng đầu của mình, bạn có thể xây dựng và kiểm thử một skill chức năng trong một buổi — thường trong 15-30 phút.

Tạo skill:

- Tạo skill từ mô tả ngôn ngữ tự nhiên
- Tạo SKILL.md được định dạng đúng với frontmatter
- Gợi ý các cụm từ kích hoạt và cấu trúc

Xem xét skill:

- Gắn cờ các vấn đề thường gặp (mô tả mơ hồ, thiếu kích hoạt, vấn đề cấu trúc) •
- Xác định rủi ro kích hoạt quá mức/không đủ
- Gợi ý các trường hợp kiểm thử dựa trên mục đích đã nêu của skill

Cải tiến lặp:

- Sau khi sử dụng skill và gặp các trường hợp biên hoặc lỗi, mang những ví dụ đó trở lại skill-creator
- Ví dụ: "Dùng các vấn đề & giải pháp được xác định trong chat này để cải thiện cách skill xử lý [trường hợp biên cụ thể]"

Để sử dụng:

"Dùng skill skill-creator để giúp tôi xây dựng skill cho [trường hợp sử dụng của bạn]"

Lưu ý: skill-creator giúp bạn thiết kế và cải thiện skill nhưng không thực thi các bộ kiểm thử tự động hay tạo kết quả đánh giá định lượng.

Lặp lại dựa trên phản hồi

Skill là tài liệu sống. Lên kế hoạch lặp lại dựa trên:

Tín hiệu kích hoạt không đủ:

- Skill không tải khi nên tải
- Người dùng kích hoạt thủ công
- Câu hỏi hỗ trợ về khi nào sử dụng

Giải pháp: Thêm chi tiết và sắc thái vào phần description — điều này có thể bao gồm các từ khóa đặc biệt cho các thuật ngữ kỹ thuật.

Tín hiệu kích hoạt quá mức:

- Skill tải cho các truy vấn không liên quan
- Người dùng tắt nó
- Nhầm lẫn về mục đích

Giải pháp: Thêm kích hoạt âm, cụ thể hơn.

Vấn đề thực thi:

- Kết quả không nhất quán
- Lỗi lệnh gọi API
- Cần người dùng sửa

Giải pháp: Cải thiện hướng dẫn, thêm xử lý lỗi.

Chương 4 – Phân Phối và Chia Sẻ

Skill giúp tích hợp MCP của bạn hoàn chỉnh hơn. Khi người dùng so sánh các connector, những connector có skill cung cấp con đường nhanh hơn đến giá trị, mang lại lợi thế so với các giải pháp chỉ có MCP.

Mô Hình Phân Phối Hiện Tại (Tháng 1/2026)

Cách người dùng cá nhân nhận skill:

1. Tải xuống thư mục skill
2. Nén thư mục thành .zip (nếu cần)
3. Upload lên Claude.ai qua Settings > Capabilities > Skills
4. Hoặc đặt vào thư mục skills của Claude Code

Skill cấp tổ chức:

- Quản trị viên có thể triển khai skill toàn không gian làm việc (ra mắt 18/12/2025) •
Cập nhật tự động
- Quản lý tập trung

Tiêu Chuẩn Mở

Anthropic đã công bố Agent Skills như một tiêu chuẩn mở. Giống như MCP, chúng tôi tin rằng skill nên có thể di động qua các công cụ và nền tảng — cùng một skill nên hoạt động bất kể bạn đang dùng Claude hay các nền tảng AI khác. Điều đó nói lên, một số skill được thiết kế để tận dụng đầy đủ các khả năng của một nền tảng cụ thể; tác giả có thể ghi chú điều này trong trường compatibility của skill. Chúng tôi đã hợp tác với các thành viên trong hệ sinh thái về tiêu chuẩn này, và rất vui mừng trước sự chấp nhận ban đầu.

Sử Dụng Skill Qua API

Đối với các trường hợp sử dụng lập trình — chẳng hạn như xây dựng ứng dụng, agent, hoặc quy trình tự động tận dụng skill — API cung cấp quyền kiểm soát trực tiếp đối với quản lý và thực thi skill.

Khả năng chính:

- Endpoint `/v1/skills` để liệt kê và quản lý skill
- Thêm skill vào yêu cầu Messages API qua tham số `container.skills`
- Kiểm soát phiên bản và quản lý qua Claude Console
- Hoạt động với Claude Agent SDK để xây dựng agent tùy chỉnh

Khi nào nên dùng skill qua API so với Claude.ai:

| Trường Hợp Sử Dụng | Nền Tảng Tốt Nhất |
|---|-------------------------|
| Người dùng cuối tương tác trực tiếp với skill | Claude.ai / Claude Code |

| | |
|---|-------------------------|
| Kiểm thử thử công và lập trong quá trình phát triển | Claude.ai / Claude Code |
| Quy trình cá nhân, không thường xuyên | Claude.ai / Claude Code |
| Ứng dụng dùng skill theo lập trình | API |
| Triển khai production quy mô lớn | API |
| Pipeline tự động và hệ thống agent | API |

Lưu ý: Skill trong API yêu cầu beta Code Execution Tool, cung cấp môi trường an toàn mà skill cần để chạy.

Để biết chi tiết triển khai, xem:

[Skills API Quickstart](#)

[Tạo Custom Skills](#)

[Skills trong Agent SDK](#)

Cách Tiếp Cận Được Khuyến Nghị Hiện Nay

Bắt đầu bằng cách lưu trữ skill trên GitHub với repo công khai, README rõ ràng (dành cho người đọc — đây khác với thư mục skill, không nên chứa README.md), và ví dụ sử dụng kèm ảnh chụp màn hình. Sau đó thêm một phần vào tài liệu MCP của bạn liên kết đến skill, giải thích lý do tại sao sử dụng cả hai cùng nhau có giá trị, và cung cấp hướng dẫn bắt đầu nhanh.

5. Lưu trữ trên GitHub

- Repo công khai cho skill mã nguồn mở
- README rõ ràng với hướng dẫn cài đặt
- Ví dụ sử dụng và ảnh chụp màn hình

6. Tài liệu trong Repo MCP

- Liên kết đến skill từ tài liệu MCP
- Giải thích giá trị khi dùng cả hai cùng nhau
- Cung cấp hướng dẫn bắt đầu nhanh

7. Tạo Hướng Dẫn Cài Đặt

Cài đặt Skill [Tên Dịch Vụ]

1. Tải skill:

- Clone repo: `git clone https://github.com/yourcompany/skills` -

Hoặc tải ZIP từ Releases

2. Cài vào Claude:

- Mở Claude.ai > Settings > Skills

- Nhấp "Upload skill"

- Chọn thư mục skill (đã nén)

3. Bật skill:

- Bật [Tên Dịch Vụ] skill

- Đảm bảo MCP server đã kết nối

4. Kiểm thử:

- Hỏi Claude: "Thiết lập dự án mới trong [Tên Dịch Vụ]"

Định Vị Skill Của Bạn

Cách bạn mô tả skill quyết định liệu người dùng có hiểu giá trị và thực sự thử nó hay không. Khi viết về skill — trong README, tài liệu, hoặc marketing — hãy ghi nhớ các nguyên tắc sau:

Tập trung vào kết quả, không phải tính năng:

✅ Tốt:

"Skill ProjectHub cho phép nhóm thiết lập không gian làm việc dự án hoàn chỉnh trong vài giây — bao gồm trang, cơ sở dữ liệu và template — thay vì mất 30 phút thiết lập thủ công."

❌ Không tốt:

"Skill ProjectHub là một thư mục chứa YAML frontmatter và hướng dẫn Markdown gọi các công cụ MCP server của chúng tôi."

Nêu bật câu chuyện MCP + skills:

"MCP server của chúng tôi cung cấp cho Claude quyền truy cập vào các dự án Linear của bạn."

Skill của chúng tôi dạy Claude quy trình lập kế hoạch sprint của nhóm bạn.

Cùng nhau, chúng cho phép quản lý dự án được hỗ trợ bởi AI."

Chương 5 – Mô Hình và Xử Lý Sự Cố

Các mô hình này xuất hiện từ các skill được tạo bởi những người dùng sớm và các nhóm nội bộ. Chúng đại diện cho các cách tiếp cận phổ biến đã được chứng minh hoạt động tốt — không phải template quy định bắt buộc.

Chọn cách tiếp cận: Ưu tiên vấn đề hay công cụ

Hãy nghĩ như Home Depot. Bạn có thể bước vào với một vấn đề — "Tôi cần sửa tủ bếp" — và nhân viên chỉ bạn đến đúng công cụ. Hoặc bạn có thể chọn một máy khoan mới và hỏi cách sử dụng nó cho công việc cụ thể của bạn.

Skill hoạt động theo cách tương tự:

- Ưu tiên vấn đề: "Tôi cần thiết lập không gian dự án" → Skill của bạn điều phối các lệnh gọi MCP đúng theo thứ tự đúng. Người dùng mô tả kết quả; skill xử lý công cụ.
- Ưu tiên công cụ: "Tôi có Notion MCP kết nối" → Skill của bạn dạy Claude các quy trình và thực tiễn tốt nhất tối ưu. Người dùng có quyền truy cập; skill cung cấp chuyên môn.

Hầu hết skill nghiêng về một hướng. Biết cách định khung phù hợp với trường hợp của bạn giúp chọn đúng mô hình dưới đây.

Mô Hình 1: Điều Phối Quy Trình Tuần Tự

Dùng khi: Người dùng cần các quy trình nhiều bước theo thứ tự cụ thể.

Cấu trúc ví dụ:

```
## Quy Trình: Onboard Khách Hàng Mới
```

```
### Bước 1: Tạo Tài Khoản
```

```
Gọi MCP tool: `create_customer`
```

```
Tham số: name, email, company
```

```
### Bước 2: Thiết Lập Thanh Toán
```

```
Gọi MCP tool: `setup_payment_method`
```

```
Chờ: xác minh phương thức thanh toán
```

```
### Bước 3: Tạo Đăng Ký
```

```
Gọi MCP tool: `create_subscription`
```

```
Tham số: plan_id, customer_id (từ Bước 1)
```

```
### Bước 4: Gửi Email Chào Mừng
```

```
Gọi MCP tool: `send_email`
```

```
Template: welcome_email_template
```

Các kỹ thuật chính:

- Thứ tự bước rõ ràng
- Phụ thuộc giữa các bước
- Xác thực ở mỗi giai đoạn
- Hướng dẫn rollback khi gặp lỗi

Mô Hình 2: Điều Phối Đa MCP

Dùng khi: Quy trình trải rộng qua nhiều dịch vụ.

Ví dụ: Handoff thiết kế sang phát triển

Giai Đoạn 1: Xuất Thiết Kế (Figma MCP) 1.

Xuất tài sản thiết kế từ Figma

2. Tạo đặc tả thiết kế

3. Tạo manifest tài sản

Giai Đoạn 2: Lưu Trữ Tài Sản (Drive MCP)

1. Tạo thư mục dự án trong Drive

2. Upload tất cả tài sản

3. Tạo link chia sẻ

Giai Đoạn 3: Tạo Task (Linear MCP)

1. Tạo các task phát triển

2. Đính kèm link tài sản vào task

3. Giao cho nhóm kỹ thuật

Giai Đoạn 4: Thông Báo (Slack MCP) 1.

Post tóm tắt handoff lên #engineering 2. Bao

gồm link tài sản và tham chiếu task

Các kỹ thuật chính:

- Phân tách giai đoạn rõ ràng
- Truyền dữ liệu giữa các MCP
- Xác thực trước khi chuyển sang giai đoạn tiếp theo •

Xử lý lỗi tập trung

Mô Hình 3: Cải Tiến Lặp

Dùng khi: Chất lượng đầu ra cải thiện qua nhiều lần lặp.

Ví dụ: Tạo báo cáo

Tạo Báo Cáo Lặp

Bản Nháp Đầu

1. Lấy dữ liệu qua MCP

2. Tạo bản nháp báo cáo đầu tiên

3. Lưu vào file tạm

Kiểm Tra Chất Lượng

1. Chạy script xác thực: `scripts/check\_report.py`

2. Xác định vấn đề:

- Thiếu phần
- Định dạng không nhất quán
- Lỗi xác thực dữ liệu

Vòng Lặp Cải Tiến

1. Giải quyết từng vấn đề được xác định

2. Tạo lại các phần bị ảnh hưởng

3. Xác thực lại

4. Lặp lại cho đến khi đạt ngưỡng chất lượng

Hoàn Thiện

1. Áp dụng định dạng cuối cùng

2. Tạo tóm tắt

3. Lưu phiên bản cuối

Các kỹ thuật chính:

- Tiêu chí chất lượng rõ ràng
- Cải tiến lặp
- Script xác thực
- Biết khi nào dừng lặp

Mô Hình 4: Lựa Chọn Công Cụ Theo Ngữ Cảnh Dừng khi: Cùng kết quả nhưng dùng công cụ khác nhau tùy ngữ cảnh. Ví dụ: Lưu trữ file

Lưu Trữ File Thông Minh

Cây Quyết Định

1. Kiểm tra loại file và kích thước
2. Xác định vị trí lưu trữ tốt nhất:
 - File lớn (>10MB): Dùng cloud storage MCP
 - Tài liệu cộng tác: Dùng Notion/Docs MCP
 - File code: Dùng GitHub MCP
 - File tạm: Dùng local storage

Thực Thi Lưu Trữ

Dựa trên quyết định:

- Gọi MCP tool phù hợp
- Áp dụng metadata đặc thù dịch vụ
- Tạo link truy cập

Cung Cấp Ngữ Cảnh cho Người Dừng

Giải thích lý do tại sao chọn vị trí lưu trữ đó

Các kỹ thuật chính:

- Tiêu chí quyết định rõ ràng
- Các tùy chọn dự phòng
- Minh bạch về lựa chọn

Mô Hình 5: Trí Tuệ Đặc Thù Lĩnh Vực

Dừng khi: Skill bổ sung kiến thức chuyên biệt ngoài quyền truy cập công cụ.

Ví dụ: Tuân thủ tài chính

Xử Lý Thanh Toán Tuân Thủ

Trước Xử Lý (Kiểm Tra Tuân Thủ)

1. Lấy chi tiết giao dịch qua MCP
2. Áp dụng quy tắc tuân thủ:
 - Kiểm tra danh sách trừng phạt
 - Xác minh cho phép theo thẩm quyền
 - Đánh giá mức độ rủi ro
3. Ghi lại quyết định tuân thủ

Xử Lý

NẾU tuân thủ được thông qua:

- Gọi MCP tool xử lý thanh toán
- Áp dụng kiểm tra gian lận phù hợp
- Xử lý giao dịch

KHÔNG THÌ:

- Gắn cờ để xem xét
- Tạo case tuân thủ

Dấu Vết Kiểm Toán

- Ghi lại tất cả kiểm tra tuân thủ
- Ghi lại quyết định xử lý
- Tạo báo cáo kiểm toán

Các kỹ thuật chính:

- Chuyên môn lĩnh vực được nhúng trong logic
- Tuân thủ trước khi hành động
- Tài liệu toàn diện
- Quản trị rõ ràng

Xử Lý Sự Cố

Skill không upload được

Lỗi: "Could not find SKILL.md in uploaded folder"

Nguyên nhân: File không được đặt tên chính xác là SKILL.md

Giải pháp:

- Đổi tên thành SKILL.md (phân biệt chữ hoa/thường)
- Xác minh bằng: ls -la phải hiển thị SKILL.md

Lỗi: "Invalid frontmatter"

Nguyên nhân: Vấn đề định dạng YAML

Lỗi thường gặp:

```
# Sai - thiếu dấu phân cách
name: my-skill
description: Làm gì đó
```

```
# Sai - nháy kép không đóng
name: my-skill
description: "Làm gì đó"
```

```
# Đúng
---
name: my-skill
description: Làm gì đó
---
```

Lỗi: "Invalid skill name"

Nguyên nhân: Tên có khoảng cách hoặc chữ hoa

```
# Sai
name: My Cool Skill
```

```
# Đúng
name: my-cool-skill
```

Skill không kích hoạt

Triệu chứng: Skill không bao giờ tải tự động

Cách khắc phục: Sửa lại trường description. Danh sách kiểm tra nhanh:

- Có quá mơ hồ không? ("Giúp đỡ với dự án" sẽ không hoạt động)
- Có bao gồm các cụm từ kích hoạt mà người dùng thực sự sẽ nói không? •
- Có đề cập đến loại file liên quan không?

Cách debug:

Hỏi Claude: "Bạn sẽ dùng skill [tên skill] khi nào?" Claude sẽ trích dẫn lại phần description. Điều chỉnh dựa trên những gì còn thiếu.

Skill kích hoạt quá thường xuyên

Triệu chứng: Skill tải cho các truy vấn không liên quan

Giải pháp:

8. Thêm kích hoạt âm

description: Phân tích dữ liệu nâng cao cho file CSV. Dùng cho mô hình thống kê, hồi quy, phân cụm. KHÔNG dùng cho khám phá dữ liệu đơn giản (dùng skill data-viz thay thế).

9. Cụ thể hơn

Quá rộng

description: Xử lý tài liệu

Cụ thể hơn

description: Xử lý tài liệu PDF pháp lý để rà soát hợp đồng

10. Làm rõ phạm vi

description: Xử lý thanh toán PayFlow cho thương mại điện tử. Dùng cụ thể cho quy trình thanh toán online, không dùng cho các truy vấn tài chính chung.

Vấn đề kết nối MCP

Triệu chứng: Skill tải nhưng các lệnh gọi MCP thất bại

Danh sách kiểm tra:

11. Xác minh MCP server được kết nối

- Claude.ai: Settings > Extensions > [Dịch Vụ Của Bạn]
- Phải hiển thị trạng thái "Connected"

12. Kiểm tra xác thực

- API key hợp lệ và chưa hết hạn
- Quyền/scope phù hợp được cấp
- Token OAuth đã được làm mới

13. Kiểm thử MCP độc lập

- Yêu cầu Claude gọi MCP trực tiếp (không có skill)
- "Dùng [Dịch Vụ] MCP để lấy dự án của tôi"
- Nếu thất bại, vấn đề là MCP không phải skill

14. Xác minh tên công cụ

- Skill tham chiếu đúng tên công cụ MCP
- Kiểm tra tài liệu MCP server
- Tên công cụ phân biệt chữ hoa/thường

Hướng dẫn không được tuân theo

Triệu chứng: Skill tải nhưng Claude không tuân theo hướng dẫn

Nguyên nhân thường gặp:

15. Hướng dẫn quá dài

- Giữ hướng dẫn ngắn gọn
- Dùng gạch đầu dòng và danh sách đánh số
- Chuyển tài liệu tham chiếu chi tiết sang các file riêng

16. Hướng dẫn bị chôn vùi

- Đặt hướng dẫn quan trọng ở đầu
 - Dùng các tiêu đề ## Quan Trọng hoặc ## Cực Kỳ Quan Trọng •
- Lặp lại các điểm chính nếu cần

17. Ngôn ngữ mơ hồ

Không tốt

Hãy chắc chắn xác thực mọi thứ đúng cách

Tốt

CỰC KỲ QUAN TRỌNG: Trước khi gọi create_project, xác minh:

- Tên dự án không trống
- Ít nhất một thành viên nhóm được giao
- Ngày bắt đầu không phải trong quá khứ

18. Mô hình "lười biếng"

Thêm khuyến khích rõ ràng:

```
## Lưu Ý Hiệu Suất
- Hãy dành thời gian để làm điều này kỹ lưỡng
- Chất lượng quan trọng hơn tốc độ
- Không bỏ qua các bước xác thực
```

Lưu ý: Thêm điều này vào prompt người dùng hiệu quả hơn so với trong SKILL.md.

Vấn đề ngữ cảnh lớn

Triệu chứng: Skill có vẻ chậm hoặc phản hồi bị suy giảm

Nguyên nhân:

- Nội dung skill quá lớn
- Quá nhiều skill được bật cùng lúc
- Tất cả nội dung được tải thay vì dùng hiển thị dần dần

Giải pháp:

19. Tối ưu kích thước SKILL.md

- Chuyển tài liệu chi tiết sang references/
- Liên kết đến references thay vì inline
- Giữ SKILL.md dưới 5.000 từ

20. Giảm skill được bật

- Đánh giá nếu bạn có hơn 20–50 skill được bật cùng lúc
- Khuyến nghị bật có chọn lọc
- Cân nhắc "gói skill" cho các khả năng liên quan

Chương 6 – Tài Nguyên và Tham Khảo

Nếu bạn đang xây dựng skill đầu tiên, hãy bắt đầu với Hướng Dẫn Thực Tiễn Tốt Nhất, sau đó tham khảo tài liệu API khi cần.

Tài Liệu Chính Thức

Tài nguyên Anthropic:

- Hướng Dẫn Thực Tiễn Tốt Nhất
- Tài Liệu Skills
- Tham Khảo API
- Tài Liệu MCP

Bài viết Blog:

- Giới Thiệu Agent Skills
- Blog Kỹ Thuật: Trang Bị Agents cho Thế Giới Thực
- Skills Explained
- Cách Tạo Skills cho Claude
- Xây Dựng Skills cho Claude Code
- Cải Thiện Thiết Kế Frontend thông qua Skills

Công Cụ và Tiện Ích

Skill skill-creator:

- Tích hợp sẵn trong Claude.ai và có sẵn cho Claude Code
- Có thể tạo skill từ mô tả
- Xem xét và đưa ra khuyến nghị
- Sử dụng: "Giúp tôi xây dựng skill bằng skill-creator"

Xác thực:

- skill-creator có thể đánh giá skill của bạn
- Hỏi: "Xem xét skill này và đề xuất cải tiến"

Phụ Lục A: Danh Sách Kiểm Tra Nhanh

Sử dụng danh sách kiểm tra này để xác thực skill của bạn trước và sau khi upload. Nếu bạn muốn bắt đầu nhanh hơn, hãy dùng skill skill-creator để tạo bản nháp đầu tiên, sau đó chạy qua danh sách này để đảm bảo bạn không bỏ sót gì.

Trước khi bắt đầu

- Đã xác định 2–3 trường hợp sử dụng cụ thể
- Đã xác định công cụ (tích hợp sẵn hoặc MCP)
- Đã xem lại hướng dẫn này và các skill ví dụ
- Đã lên kế hoạch cấu trúc thư mục

Trong quá trình phát triển

- Thư mục được đặt tên theo kebab-case
- File SKILL.md tồn tại (đúng chính tả)
- YAML frontmatter có dấu phân cách ---
- Trường name: kebab-case, không có khoảng cách, không có chữ hoa • ☐
- description bao gồm CHỨC NĂNG và KHI NÀO dùng
- Không có thẻ XML (< >) ở bất kỳ đâu
- Hướng dẫn rõ ràng và có thể thực hiện được
- Xử lý lỗi được bao gồm
- Các ví dụ được cung cấp
- Tài liệu tham chiếu được liên kết rõ ràng

Trước khi upload

- Đã kiểm thử kích hoạt trên các nhiệm vụ rõ ràng
- Đã kiểm thử kích hoạt trên các yêu cầu được diễn đạt lại
- Đã xác minh không kích hoạt trên các chủ đề không liên quan
- Kiểm thử chức năng đã qua
- Tích hợp công cụ hoạt động (nếu có)
- Đã nén thành file .zip

Sau khi upload

- Kiểm thử trong các cuộc hội thoại thực tế
- Giám sát kích hoạt không đủ/quá mức
- Thu thập phản hồi người dùng
- Lập lại trên description và hướng dẫn
- Cập nhật phiên bản trong metadata

Phụ Lục B: YAML Frontmatter

Các trường bắt buộc

```
---
name: ten-skill-kebab-case
description: Skill làm gì và khi nào nên dùng. Bao gồm các cụm từ kích hoạt cụ thể. -
--
```

Tất cả trường tùy chọn

```
name: skill-name
description: [mô tả bắt buộc]
license: MIT # Tùy chọn: Giấy phép cho mã nguồn mở
allowed-tools: "Bash(python:*) Bash(npm:*) WebFetch" # Tùy chọn: Giới hạn truy cập công cụ
metadata: # Tùy chọn: Các trường tùy chỉnh
  author: Company Name
  version: 1.0.0
  mcp-server: server-name
  category: productivity
  tags: [project-management, automation]
  documentation: https://example.com/docs
```

Ghi chú bảo mật

Được phép:

- Bất kỳ kiểu YAML tiêu chuẩn nào (chuỗi, số, boolean, danh sách, đối tượng) •
- Các trường metadata tùy chỉnh
- Mô tả dài (tối đa 1024 ký tự)

Bị cấm:

- Dấu ngoặc nhọn XML (< >) — hạn chế bảo mật
- Thực thi mã trong YAML (dùng phân tích cú pháp YAML an toàn)
- Skill đặt tên có tiền tố "claude" hoặc "anthropic" (đã được đặt trước)

Phụ Lục C: Ví Dụ Skill Hoàn Chỉnh

Để có các skill đầy đủ, sẵn sàng production minh họa các mô hình trong hướng dẫn này:

- Skill Tài Liệu — Tạo PDF, DOCX, PPTX, XLSX: anthropics/skills
- Skill Ví Dụ — Nhiều mô hình quy trình: anthropics/skills/tree/main/example
- Thư Mục Skill Đối Tác — Xem skill từ Asana, Atlassian, Canva, Figma, Sentry, Zapier và nhiều hơn: anthropics/skills/tree/main/partner

Các kho lưu trữ này luôn được cập nhật và bao gồm các ví dụ bổ sung ngoài những gì được đề cập ở đây. Hãy clone chúng, sửa đổi cho trường hợp sử dụng của bạn, và dùng làm template.

claude.ai